Verified-Answer Precision as the Acceptance Test for Speed and Quantization

A Qwen3.8-Flash-Next Tool Agent for ARC-AGI-2 on Four L4 GPUs

Md. Hamid Hosen
Dept. of CSE, East Delta University
mdhamidhosen4@gmail.com

MD. Foysal Emon Shanto
Dept. of CSE, Jashore University of Science and Technology
foyshalsk348@gmail.com

Md Esfer Abdus Sami
Dept. of CSE, North South University
esfer.sami.me@gmail.com

Team Hack2Publish, ARC Prize 2026

November 2026

Abstract

A faster ARC-AGI-2 system, from a compressed model or more concurrent sessions, reaches more tasks. A tool agent, however, grades its own answers: a program that reproduces every demonstration pair counts as verified, and a weaker model passes that test with the wrong rule more often. We therefore judge a speed change by the *correct* verified answers it yields on the tasks the deployed system reaches, with verification precision as a guard. Our submission 57851995 (public leaderboard 54.17) runs Qwen3.8-Flash-Next offline on Kaggle's 4×L4 with 4-bit routed experts, an FP8 KV cache and speculative decoding, driving up to nine sessions of a Python tool agent. Against a faster mixed 3/2-bit build of the same agent, on the 92 evaluation tasks both started in 6-hour runs, each build verified 79 first attempts, but 61 versus 50 were correct (Fisher $p = 0.08$; $p = 0.043$ for host-accepted submissions). Most false verifications fit rules the demonstrations leave open. The 3-bit build reached more tasks and won on evaluation (65.0 vs 53.75) yet scored lower on the leaderboard (49.58). FP8 KV cost no measured accuracy and nearly doubled concurrency. We also report evaluation-versus-leaderboard inversions and a ledger of negative results; the precision finding replicates another 2026 team's observation.

1 Introduction

ARC-AGI measures the ability to acquire a new skill from a handful of examples [5, 7]. Each task shows two to five input/output grid pairs; the solver must produce the exact output grid for one or two test inputs, with two attempts each. The tasks are easy for people and hard for current AI systems, which makes the benchmark a probe of generalisation rather than of memorised knowledge [6]. The ARC Prize 2026 competition runs a submitted notebook on 240 hidden tasks within 12 hours on four 24 GB NVIDIA L4 GPUs, without internet access.

Language models that write and test programs are the strongest approach on this hardware in 2026 [13, 3, 20]. The score is roughly the product of *coverage* (the share of tasks that get a full attempt) and the *solve rate* of an attempt. Both draw on the same budget: compression or more concurrency raises coverage but can lower the quality of each attempt.

Our approach grew out of our own leaderboard history (Figure 1): six months of test-time-training reproductions stayed at or below 32.64, a dense 27B model reached 34.58, and a sparse Flash-Next agent reached 54.17. A faster 3-bit version of the same agent covered more of the evaluation set yet scored lower on the leaderboard. This paper explains why and turns it into a rule.

Contributions.

1. **A decision rule with matched evidence.** Judge a speed or compression change by correct verified answers at the deployment's time per task and task order, and require that verification precision does not fall on the tasks both configurations reach. We measure precision for W4A16 and 3-bit builds on matched tasks with confidence intervals (Section 5.2) and classify every train-verified wrong program (Table 4).
2. **A coverage-versus-precision decomposition** of two 6-hour runs: where the faster build wins (tasks the slower one never reaches) and where it loses (the tasks both reach).
3. **Three evaluation-versus-leaderboard inversions**, including a thinking-cap inversion for a dense 27B model and one between two public notebooks run by us under equal windows.
4. **A negative-results ledger and a union oracle** that bound what complementary systems could add, and a reproducible low-throughput recipe (Section 4, Appendix A).

2 Related work

Test-time training. The ARChitects [11, 12] and NVARC [24] fine-tune a small model per task and rank sampled grids by augmentation scores; Akyürek et al. [1] studied test-time training on ARC. Our companion paper [15] decomposes our NVARC-derived runs into generation and selection.

Program synthesis with verification. Greenblatt [13] sampled many programs and kept those that reproduce the demonstrations; Li et al. [17] combined induction and transduction; SOAR [22] evolves programs with self-improving models; Poetiq [21] refines code with training-pair feedback; pivotools [3] showed that an agentic Python loop improves ARC-AGI-2 scores across models. The Duck harness [2] won ARC-AGI-3 Milestone 1 with a persistent Python tool; our agent uses Franzen's fork of it [10] with an ARC-AGI-2 adapter.

2026 Flash-Next systems on 4×L4. Several teams published writeups with the same model and hardware (Table 1). notnitsuj [20] reports that compression shows up as less precise verified

System (writeup)	Public LB	tok/s	Sessions	Compression; selection
Yi-Chia Chen [4]	87.22	1,240	128	320 experts INT3, distillation; solvers + refiner
Domdolus [8]	83.47	–	60–70	2-bit experts, 4-bit KV; train-pass gate
fit-the-whale [9]	81.81	–	–	DeepSeek-V4.1-Flash (different model)
Majkel [18]	78.89	≈700	28	NVFP4, 352 experts; train-pass gate
nisshokuitsuki [19]	77.22–78.06	≈680	≤64	W4A16, 448 experts; backtest + rule checks
Sypetkowski et al. [25]	77.36	≈700	–	3-bit, INT4 KV; two passes + vote
notnitsuj [20]	75.69	1,087–1,120	48	320 experts 4/3-bit; invariants + voting
This paper	**54.17**	**172–225**	**9**	**W4A16, all 512 experts, FP8 KV; train-pass gate**

Table 1: Public 2026 systems on the same hardware, as reported in their writeups ("–" = not reported in comparable form). Ours is the low-throughput, near-stock-engine, no-pruning, no-training point.

programs rather than fewer of them, that memory sets throughput, and that aborting slow sessions costs points. Our precision result is an independent replication with a different pair of precisions (W4A16 vs mixed W3/W2 AutoRound experts at about 3 bits per weight, all 512 experts, no pruning or training) on matched tasks.

3 The path to the submission

April–September: test-time training. About 70 submissions built on public notebooks, mostly of the NVARC 2025 lineage, scored 23.75–32.64. Our companion paper [15] shows that in two logged evaluation runs 116–117 of 172 test inputs had no correct candidate above the search threshold, while perfect selection could add 4.67–7.67 points.

October: Qwen3.8-27B with code verification. A Poetiq-style solver that writes code and checks it on the training pairs reached 34.58 with 60k thinking tokens per attempt. The same solver family with a 77.8k cap scored 48.61 % on the evaluation set but 21.39 on the leaderboard, and a 49k cap gave 32.08. A replay simulation of our logs estimated that about 208 of 240 hidden tasks receive a first iteration at 60k against about 177 at 77.8k. These notebooks differ in more than the cap, so we read the 13-point gap as consistent with lost coverage, not as a controlled result.

October: Qwen3.8-27B as a tool agent. A Duck-style agent on the same model scored 17.50–22.92, below the code-verifying solver. In our first agent runs (per-response cap 49k) the first call to the checker came at a median of about 55k tokens; in later 120-task runs it came at 25–31k, still most of an attempt budget of 35–45k. The dense model reasoned at length before testing anything.

November: Qwen3.8-Flash-Next. The same agent on the sparse model scored 54.17 (W4A16, this submission) and 49.58 (3-bit "klee100" build).

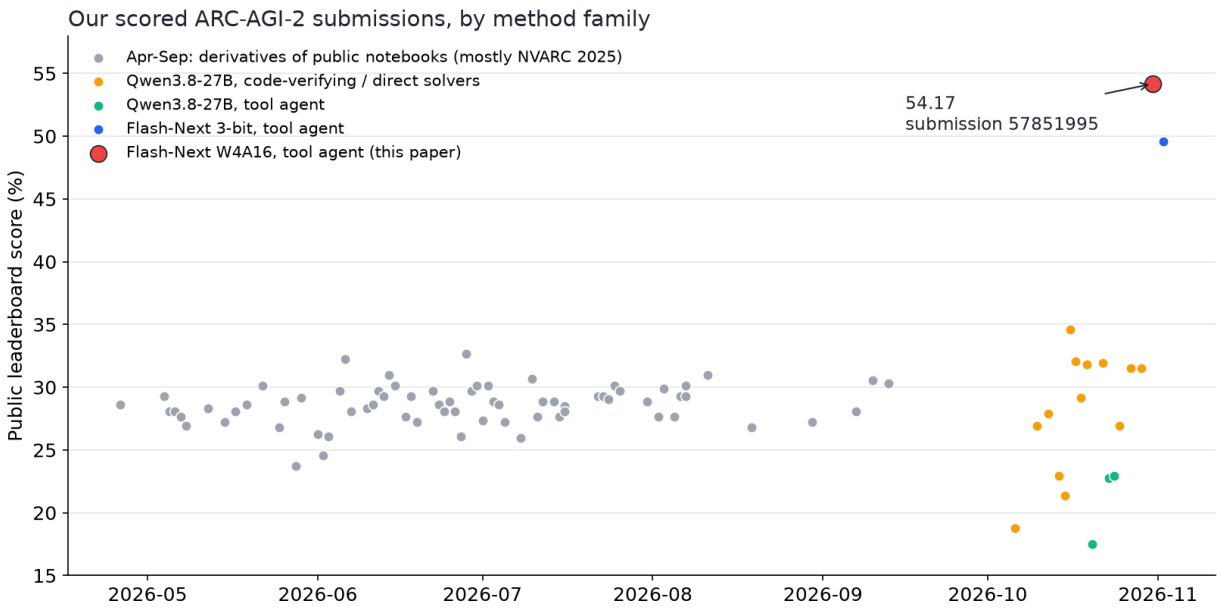


Figure 1: Public leaderboard scores of our team's scored submissions (runs below 5 omitted). Repeated runs of one notebook vary with a standard deviation of about 1.5 points.

4 System

Model and engine. Qwen3.8-Flash-Next [23] is a 48-layer mixture-of-experts model with 512 routed experts per layer, top-10 routing, multi-token-prediction heads and a large n-gram embedding table. We serve it with Merlin v3.0.0 [14], a vLLM-0.32 fork [16], using a W4A16 checkpoint (routed experts in 4 bits), an FP8 copy of the n-gram embedding table, an FP8 E4M3 KV cache (pool 738,142 tokens), tensor parallelism 2 × pipeline parallelism 2 with expert parallelism, prefix caching, and MTP speculation with 3 draft tokens and probabilistic acceptance. If the engine fails to boot, the KV cache size steps down (at most three tries). Up to nine agent sessions run at once.

Agent. Each attempt is one session with a single `python` tool in a sandbox (30 s per call, 4 GB address space). Raw grids are never in the prompt: the model sees a compact "facts" view (shapes, colour counts, changed cells, components) and reads exact grids inside Python. The loop is: reason, write `transform(grid)`, call `check()` (exact pairs, and up to six wrong cells per failing pair), revise, and call `submit()`, which the host re-verifies in a clean sandbox. Sampling: temperature 1.0, top-p 0.95, top-k 20, reasoning effort xhigh, 104k tokens per attempt.

Scheduling and answers. Attempt 1 runs for every task first, in a priority order (tasks with at least three training pairs and a single test input first, then smaller grids). Attempt 2 is an independent session. Each test input is filled separately: train-verified programs first (agreeing candidates before single ones), then partially fitting programs, then a small rule library, then the test input itself. A complete `submission.json` with an identity fallback is written at notebook start and rewritten after every finished attempt, so unverified outputs are submitted when nothing verifies.

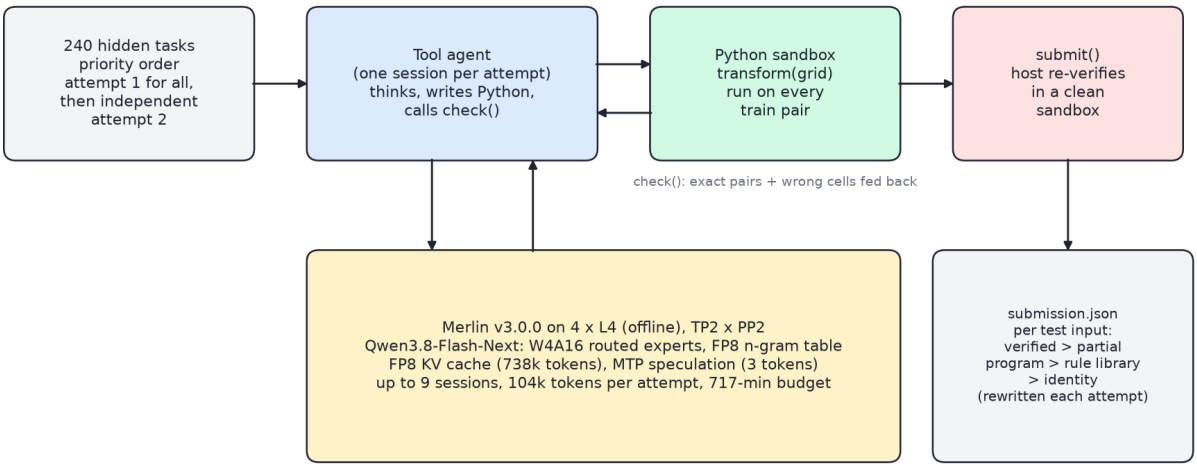


Figure 2: Pipeline of submission 57851995. Details in Appendix A.

5 Results

5.1 Leaderboard

Table 2 gives our best public leaderboard score in each method family. Each is a single run; repeated runs of one notebook vary with a standard deviation of about 1.5 points.

Method family (best submission)	Public LB (%)
NVARC-lineage test-time training	32.64
Qwen3.8-27B, code-verifying solver (60k)	34.58
Qwen3.8-27B, tool agent	22.92
Flash-Next 3-bit (klee100), tool agent (57896498)	49.58
Flash-Next W4A16, tool agent (57851995)	**54.17**

Table 2: Best public leaderboard score per family; one run each (run-to-run standard deviation about 1.5).

5.2 Precision of verification

We re-scored every train-verified program against the gold outputs, counting an attempt correct only if every test output matches. In every sample the 3-bit build's verified programs were wrong more often (Table 3, Figure 3). On the 92 tasks both 6-hour runs started, the two builds verified exactly as many first attempts (79 each) but the 3-bit build was right on 11 fewer: it is "less precise, not fewer", as notnitsuj reported for a different pair of precisions [20]. With our sample sizes the difference is at the edge of significance ($p = 0.04$–0.08 depending on the definition). On the 15 matched tasks the 3-bit build also spent 1.44× the tokens, and 34.5 % of its tokens went to attempts that verified but were wrong (W4A16: 18.1 %).

Table 4 classifies all false verifications by reading the final program, the demonstrations and the gold output. Eleven are rules the demonstrations do not pin down. Nine of the thirteen 3-bit

Sample	Build	Verified	Correct	Precision (95 % CI)	Fisher p
6 h, 92 tasks both started (attempt 1)	W4A16	79	61	77.2 % (66.8–85.1)	0.08
	3-bit	79	50	63.3 % (52.3–73.1)	
6 h, all 120 tasks	3-bit	104	65	62.5 % (52.9–71.2)	0.037*
15 matched tasks, 2 attempts	W4A16	26	24	92.3 % (75.9–97.9)	0.25
	3-bit	28	22	78.6 % (60.5–89.8)	
3-bit, all 30 tasks	3-bit	54	41	75.9 % (63.1–85.4)	

Table 3: Share of train-verified programs that are correct on the test (Wilson intervals). On the 92 shared tasks, counting host-accepted submissions only gives 58/72 vs 49/76 ($p = 0.043$). *Against W4A16's 61/79; this row includes 28 tasks only the 3-bit build started. The 6-hour builds differ from the submitted ones (Section 5.4).

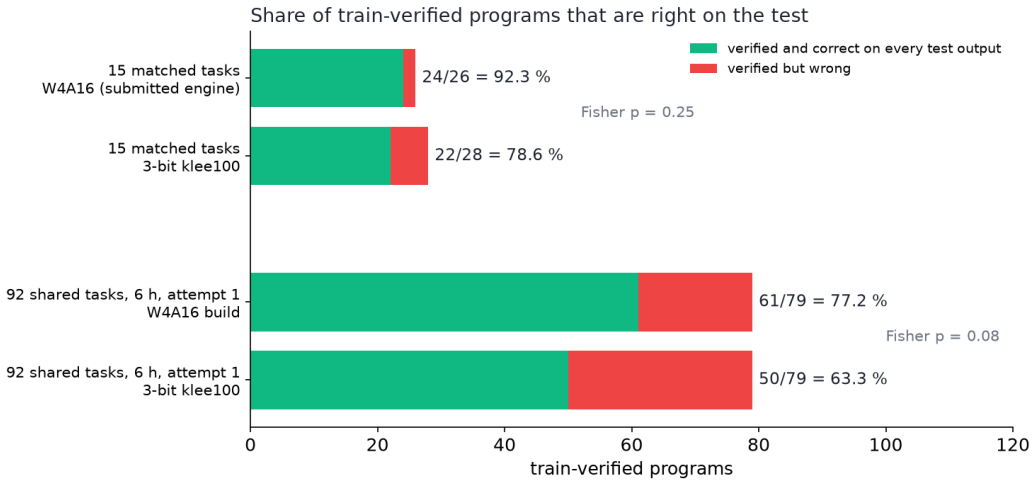


Figure 3: Train-verified submitted attempts, correct on every test output (green) or not (red).

Class	Description (example)	Count
(b) underdetermined rule	another rule fits all demonstrations but not the test (overlap priority of pasted shapes in 247ef758; cavity choice in 16b78196, two pairs)	11
(a) hard-coded structure	fixed column offsets or per-row caps tuned to the pairs; flagged by the agent's own `fixed_offset` check but not blocked	2
(c) test outside demonstrations	4×4 test grid with a new colour; all pairs 8×8 or 16×16	1
(d) output size	size from fitted arithmetic, wrong on larger test grids	1

Table 4: All 15 train-verified wrong programs from the two quality runs: 2 from W4A16 (15-task run; both class (b), with the same wrong grids as the 3-bit build) and 13 from the 3-bit 30-task run, 6 of them on the 15 matched tasks.

Build (6 h, 120 tasks)	tok/s	Tokens	Tasks started	Outputs /172	Task-norm. %
W4A16, 12 sessions	182.3	3.94 M	92	90	53.75
3-bit klee100, 24 sessions	419.4	9.06 M	120	107	65.00

Table 5: Six-hour evaluation runs.

cases are wrong in at most 8 cells, and in seven the test grid is larger than every demonstration; we did not measure how often correct verifications share these features. The two W4A16 errors are byte-identical to 3-bit errors, so on those tasks the cause is the task's ambiguity, not quantization.

5.3 Memory and concurrency

In a paired run on the same 15 tasks (same seed, equal 4,050 s windows), FP8 and BF16 KV caches each solved 13 tasks (18/20 outputs; task-normalised 86.67 vs 90.00; McNemar $p = 1.0$, 2 discordant tasks). FP8 gave 1.9× the KV pool (214 vs 112 blocks) and 1.8× the sessions (9 vs 5); the steady decode median rose from 169 to 225 tok/s, the window mean from 158 to 172 tok/s. FP8 had failed our teacher-forced KV test before this run, so teacher-forced divergence would have rejected a change that cost nothing measurable end to end. Nine sessions of up to 104k tokens need far less than 9×104k of live KV: the peak live context was 363,807 tokens (49 % of the pool) over 5 requests, with no preemption and only 3 of 270 requests sent while the server queue was non-empty, during the start-up ramp.

5.4 Coverage versus precision at six hours

Both builds ran the 120 public evaluation tasks in 6-hour windows (Table 5). These builds are not the submitted ones: the W4A16 run used 12 sessions, alphabetical order, a scheduler with re-runs and an int8 draft head; it was KV-starved (2,028 preemptions). The 3-bit run (24 sessions) is the build submitted as 57896498. On the 92 tasks both builds reached, W4A16 solved 63 and the 3-bit build 57; at equal token spend (3.0 M) W4A16 had solved 44 tasks and the 3-bit build 32. The 3-bit build's lead on evaluation comes entirely from the 28 tasks W4A16 never started, of which it solved 17. The W4A16 build was still gaining 15 tasks in its last hour. The submitted builds differ from these runs and from each other: the W4A16 submission runs up to 9 uncontended sessions in priority order (tasks with at least three pairs and one test input first) with two plain attempts, while the 3-bit submission runs 24 sessions in alphabetical order with re-runs at 120k tokens and longer request timeouts. Its leaderboard score was lower (49.58 vs 54.17, one run each), which is consistent with the precision gap but not a controlled test of it.

5.5 Evaluation as a proxy for the leaderboard

Three inversions: (1) the 3-bit build beat a 12-session W4A16 variant on evaluation (65.00 vs 53.75 task-normalised) and lost on the leaderboard to the 9-session W4A16 submission; (2) our 27B solver at a 77.8k cap scored 48.61 % on evaluation and 21.39 on the leaderboard, below its 60k version; (3) we ran two public Flash-Next notebooks by other teams on the evaluation set under equal 5 h 23 min windows: nisshokuitsuki's [19] answered 151/172 outputs and Domdolus's [8] 139/172, while their published leaderboard scores are in the opposite order (77.22–78.06 vs 83.47). Candidate reasons are the per-task budget of evaluation runs, the different difficulty of the hidden set, and use of public evaluation tasks in training; we did not test them.

6 Why it works

The demonstrations are a unit test of varying strength. A program that reproduces every demonstration encodes some rule that explains them; whether it is the intended rule depends on how many alternatives the demonstrations leave open. With two or three pairs, or a test grid larger than all of them, several programs fit, and Table 4 shows that false verification concentrates there. Our data show that the 3-bit build, which also spent 44 % more tokens per task, more often settled on a rule that fits but was not intended; we hypothesise that quantization noise makes the model less likely to prefer the simplest rule a person would pick.

Coverage and precision trade against each other. The score is approximately

$$\sum_{\text{tasks}} P(\text{reached})\, P(\text{verified} \mid \text{reached})\, P(\text{correct} \mid \text{verified})$$

plus unverified fallbacks. Section 5.4 measures the first and third factors separately: the faster build raised the first and lowered the third. Which effect wins depends on which tasks the slower build fails to reach. On the hidden set, 717 minutes at 172–225 tok/s give 7.2–9.5 M tokens after a 13–17 minute boot, about 30–40k per task, the same time per task as the 6-hour runs, where the W4A16 variant reached 92 of 120 tasks. In our 15-task run, correct first attempts needed a median of 6.3k tokens on usually-solved tasks and 23.6k on sometimes-solved ones. The submission's priority order puts tasks with many demonstrations and one test input first, so the tasks it leaves unreached are likely the hard ones; in the 6-hour runs the order was alphabetical, and the 3-bit build solved 17 of the 28 tasks the W4A16 variant never started. We think this is why coverage cost the W4A16 submission less on the hidden set than on evaluation, but we did not test it.

Memory sets concurrency. An L4 has 24 GB. 4-bit experts make the model fit; the FP8 KV cache fits 1.8× the sessions next to it without measured loss. Too many sessions for the pool cause preemption: the 12-session W4A16 variant had 2,028 preemptions in 6 hours.

Second attempts. On the 15-task runs, second attempts after a verified first attempt took 30.5 % (W4A16) and 45 % (3-bit) of the tokens; W4A16's rescued one test output (247ef758, where attempt 1 had verified a partly wrong program) and the 3-bit build's rescued none. In the 6-hour 3-bit run, later attempts after a verified first attempt used 867k tokens and rescued 9 tasks, while the 12-session W4A16 variant never reached attempt 2. Public systems with more throughput report larger gains from independent sessions and voting [18, 4].

7 What did not work, and a path forward

Negative results (27B runs unless noted):

- *Early stopping* of hopeless attempts: the best text signal reached AUC 0.61–0.64, saved 2.7–4.0 % of tokens, and out of sample lost 18 % of train-passing attempts (consistent with [20]).
- *Pause and resume* at 30–60k tokens was worse than a straight run at every budget.
- *Thinking caps* of 49k and 77.8k scored below 60k on the leaderboard (32.08 and 21.39 vs 34.58), in separate notebooks.
- *Task ordering* by difficulty predicted from the task file did not beat a simple priority order, which beat random order by 3.7–6.8 points at 50–70 % budgets in a replay simulation.
- *A dense 27B tool agent* scored below the same model with a code-verifying solver.

- *A faster 3-bit Flash-Next build* scored lower on the leaderboard than the W4A16 submission (49.58 vs 54.17; builds differ in several settings), and *12 sessions on a pool sized for 9* caused 2,028 preemptions in a 6-hour run.

Headroom from complementary systems. On the 120 public evaluation tasks our best single run answered 107 of 172 test outputs and the union of all our runs up to the deadline answered 143 (task-normalised 85.69 %). Adding the two public notebooks above gives 161 of 172 (95.28 %); 11 outputs in 8 tasks were answered by none. This is an oracle over many runs, not a reachable score, but it shows that systems fail on different tasks.

Next steps, with public evidence.

1. *Acceptance test:* measure verified precision on a fixed task set before shipping any compression or concurrency change.
2. *Targeting class (b):* invariants and symmetry tie-breaks (+2.6 and +0.54 points reported by [20]), rejecting hard-coded structure (our own `fixed_offset` flag caught both class-(a) cases but did not block them; see also [18]), and induced-rule checks [19].
3. *Fewer duplicate second attempts:* in the 6-hour run, 3-bit second attempts after an already-correct first attempt used 598k tokens; skipping attempt 2 only when an independent check agrees would keep the 9 rescues (a hypothesis).
4. *Portfolio:* route tasks without a verified program to a different solver, using the union headroom above.

8 Universality and limitations

A general rule. For any execute-and-check agent on fixed hardware (code generation with unit tests, programming by example, data-wrangling scripts, proof search with a checker), the verifier's pass rate is not its precision. Judge a compression or speed change by correct verified answers at the deployment's time per task and task order, and check that verification precision on the tasks both configurations reach does not fall. Raw verified answers per hour can mislead: in our 6-hour runs the 3-bit build produced more correct verified answers (65 vs 61) because it reached more tasks, yet lost on the leaderboard. Tokens per second and teacher-forced divergence measure neither quantity. The rule cannot protect against errors the verifier cannot see, which in ARC is the underdetermined-rule class (b); there, more demonstrations or a second, independent check are needed.

Limitations.

- Leaderboard comparisons rest on one run per build (standard deviation about 1.5 points) and builds that differ in more than precision; the submitted build was not run on all 120 evaluation tasks.
- The precision difference is at the edge of significance ($p = 0.04$–0.08 on the matched 92 tasks); the 6-hour builds differ from the submitted ones.
- Qwen3.8 models may have seen public ARC data in training; we cannot measure this.

9 Conclusion

A Qwen3.8-Flash-Next tool agent with 4-bit routed experts, an FP8 KV cache and speculation on four L4 GPUs scored 54.17 on the ARC-AGI-2 public leaderboard, about 20 points above our

dense-model and test-time-training systems. A faster 3-bit build covered more tasks but, on the tasks both reached, verified as often and was right less often, mostly where the demonstrations leave the rule open; it scored lower on the leaderboard. For systems that grade their own answers, verified-answer precision belongs next to tokens per second in every engineering decision.

Reproducibility and artifacts

Public notebook of the submission: `https://www.kaggle.com/code/foysalemonshanto/q27-xhigh-e25-nb6-submit` (version 1, submission 57851995). Engine: Merlin v3.0.0 (`https://github.com/halt95/merlin-engine`, commit `ad42e37f`). This paper and the companion paper [15] are attached as PDFs to our ARC Prize 2026 Paper Track Writeup on Kaggle.

Use of AI tools

AI coding and writing assistants were used to write analysis scripts and draft text. All numbers were taken from saved run outputs and checked against them.

A Agent and engine specification

This appendix lists the configuration that the hidden rerun of submission 57851995 executes (profile `RERUN_PROD`). The agent is the Duck tool-agent harness of Tufa Labs as forked by D. Franzen (`da-fr/arc-agi-3-solution` at commit `10882e33`, Apache-2.0), vendored byte-identical except for two switched-off-by-default patches, plus our ARC-AGI-2 adapter `arc2_duck.py`. Both ship in the payload dataset `q27-e15b-payload` and are verified by SHA-256 before use. The served model name is `q27`; the weights are Qwen3.8-Flash-Next.

A.1 Prompt and task representation

The system prompt holds the task; the first user turn is a short opener. Excerpt of the system prompt (elisions marked):

> "You are a Python coding agent solving an ARC-AGI-2 grid task. [...] The training pairs show one transformation from input grid to output grid. Infer it and implement it as a top-level function `transform(grid)` that takes and returns a rectangular list of lists of ints 0-9. It is then applied to each test input; the test outputs are unknown to everyone. [...] The exact grids are not repeated in this prompt. They are in the python tool as `train` [...] and `test_inputs` [...]. The facts below were computed from them by the harness: observations, not a rule [...]. A train-exact function is not proof that it generalizes. `submit(transform)` submits your final choice and ends this branch; submit only a function that check shows exact."

- **Grids are not printed in the prompt** (view `facts` for both attempts). The prompt carries a "facts" block computed from the exact grids: per training pair the input/output shape, colour counts, modal colour, changed-cell count and bounding box (same-shape pairs) or the shape change, connected-component counts and the largest components; per test input the shape, colours and components. The block is bounded to 800 tokens and 3,600 characters by dropping detail levels, and says what was dropped.
- The exact grids are Python lists in the sandbox (`train`, `test_inputs`). `show(g, r0, c0, r1, c1)` prints an indexed view (`r03 | 0 0 5 ...`), at most 900 cells per view and 1,000 per call; raw grid-row prints in tool output are cut by the harness.

- The prompt also states the token budget of the attempt ("about 104,000 generated tokens, thinking included"), the allowed imports (no numpy), the 30-s tool time limit and the tool-output cap.

A.2 Tools

The model has exactly one tool, `python` (a persistent sandbox; top-level functions are retained across calls). Inside it the harness provides:

- `check(transform, verbose=True)`: runs `transform` on every training pair and once on each test input. It prints `check(name): k/n train pairs exact; test inputs: ok, ...` (plus an identity flag), then for each failing pair the error with its line, or the shape mismatch, or the wrong-cell count with up to 6 samples `(row, col, got, expected)`. It returns a list of per-pair dicts (`ok`, `wrong_cells`, samples, or `error` / `shape`). A train-exact result is reported to the host, which re-runs the function in a clean sandbox and, if exact, keeps it as a candidate answer; the attempt continues.
- `check_many({label: f, ...})`: the same for several functions.
- `submit(transform)`: needs a named top-level function. The host re-runs it in a clean sandbox, one sandbox per grid, with only the retained sources and one input grid, reads the value from the sandbox's structured result, and compares it with every training output. Accepted (exact on all pairs and valid test outputs) ends the attempt; otherwise the rejection summary is returned and the attempt continues.
- Exploration helpers: `arc_overview`, `inspect_pair`, `inspect_test`, `show`. Grid functions usable inside `transform`: `grid_shape`, `color_counts`, `modal_color`, `components`, `bbox`, `crop`, `rot90`, `flip_h`, `flip_v`, `transpose`, `diff_cells`, `diff_summary`, `object_diff`.
- Auto-check: after a call that defines a one-grid function named `transform*`, `solve*`, `candidate*`, `trans`, `trans_*` or `*_alt`, the harness checks it (at most 3 per call, 24 per attempt; at most 24 host verifications of exact `check` results per attempt). `submit`/`check` sent as tool names are run as Python.

A.3 Sampling and limits

Item	Value
Temperature / top-p / top-k	1.0 / 0.95 / 20 (per request; the server's generation override is the same, with min-p 0, presence 0, repetition 1.0)
Seed	none sent
Thinking	on; `reasoning_effort = xhigh` per request and as the server's chat-template default
Attempt budget	104,000 generated tokens (attempt 1 and attempt 2), thinking included; a 60,000-token thinking floor is asserted
Output per request	remaining budget − 4,096 (reserve); further clamped to 120,384 − prompt − 2,048 (120,384 = 8,192 prompt + 104,000 + 8,192 tool allowance)
Commit turn	when ≤ 5,120 tokens remain, the harness asks for one final `transform` + `check` call; it is an ordinary thinking-on request ("variant A")
Context	server max-model-len 131,072; agent window 130,560; raw reasoning kept in history
Request timeout	3,600 s, clamped to the solve deadline

Tool steps / turns	no step cap (`LOCAL_ANALYZER_TOOL_STEPS=0`); an attempt ends on acceptance, budget, deadline or repeated failures
Sandbox	30 s per call (CPU limit 31 s), address space 4,096 MB (`RLIMIT_AS`, read back per child; not verified ⇒ solver disabled), file size 1 MB, 32 open files, standard-library subset only
Tool output	capped to about 3,072 tokens

A.4 Attempt policy and answer selection

- Two independent attempts per task (fresh agent, new conversation each), both 104,000 tokens, both with the facts view. No pause or resume, no third attempt (`extra_attempts` off); attempt 2 is not skipped when attempt 1 verified, it only needs time left.
- Stage 1: attempt 1 of *every* task, in the task order below. Stage 2: attempt 2, queued when attempt 1 of that task ends, ranked by evidence class: 0 = partial progress but no verified program, 1 = nothing, 2 = one distinct verified prediction, 3 = two or more. Accepts made after the commit text ("weak") do not count toward the class.
- End of an attempt without a verified program: (1) functions written only in the reasoning or text are extracted, run and checked (an exact one goes through host verification); (2) the best partial function (most exact training pairs, then changed-cell accuracy) is host-run once on train and test; its test output is kept as a *non-accepted* candidate.
- Selection, per test input: each attempt's best (latest accepted `submit`, else latest accepted `check`) is grouped by identical output; groups are ordered by (contains a non-weak accept, number of attempts agreeing, explicit submission, earlier attempt, latest). Then other accepted candidates (latest first), then non-accepted partial candidates (most exact pairs, then latest), then outputs of a small CPU rule library (rotations, flips, transposes, crop, scales, tilings, mirrors, a global colour map) that are exact on every training pair. `attempt_1` = first grid; `attempt_2` = next distinct grid, else the test input.
- So an *unverified* answer is submitted when no attempt verifies: the best partial program's output.
- Fallback: the test input itself (identity) for both attempts, written for every task at notebook start; a task whose selection fails also falls back to identity.
- Tasks with several test inputs: one `{attempt_1, attempt_2}` per test input, each selected independently from the same programs.

A.5 Task order (ORDER2)

Tasks are sorted by $-(\,[\#\text{train pairs} \geq 3] + [\#\text{test inputs} = 1]\,)$, then by the total number of cells (train inputs + outputs + test inputs) ascending, then by task id. Attempt 1 of all tasks follows this order.

A.6 Engine

- **Merlin** v3.0.0, a public vLLM 0.32.0 fork (`github.com/halt95/merlin-engine`, tag `merlin-v3.0.0`, commit `ad42e37f`, wheel `vllm-0.32.0+merlin.7002f56537`). It is installed offline in the notebook from our Kaggle Model `e23-merlin-env` (`install_merlin_offline.sh`); the server command comes from Merlin's own `launch.build("flash-next", "fn-quad-24g-sm89", ...)`.

- **Weights**: five W4A16 checkpoint parts (Kaggle Models `qwen3-8-flash-next-w4a16-merlin-p1..p5`) and an FP8 n-gram embedding table (PLE) overlay, merged by symlink with size checks (the rerun has no SHA gate).
- **Layout**: 4×L4, tensor parallel 2 × pipeline parallel 2 with expert parallelism; block size 4,096; max-num-batched-tokens 1,024; GPU memory utilisation 0.96.
- **KV**: FP8 E4M3 with static QSA scales (`qsa_kv_scales_262k.json`). **Pin ladder**: `--kv-cache-memory-bytes` = Merlin's L4 pin 2,479,838,700 B + 1.0 GiB = 3,553,580,524 B first, then +0.75 / +0.5 / +0 GiB. The ladder starts at +1.0 only if the #60490 patch verified, else at +0.
- **#60490**: a backport of vLLM PR #60490 (issue #60489) into Merlin's `qsa.py`: during the memory-profiling dummy run the QSA indexer allocates its peak scratch (max of the prefill logits budget and decode tokens × compressed columns × 4 B, plus the top-k workspace), so the profiler accounts for it. Applied with pinned pre/post SHA-256.
- **Speculation**: MTP, 3 draft tokens, `draft_sample_method=probabilistic`; the boot read-back checks method `mtp` and $K = 3$.
- **Prefix caching** on (`--enable-prefix-caching`, `--mamba-cache-mode align`, `--prefix-match-unit 64`); `--use-replayssm`.
- **CAPS16**: `cudagraph_mode FULL_AND_PIECEWISE` with capture sizes 1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 40, 48, 64 (FULL decode graphs cover 16 × (1+3) tokens).
- `MERLIN_SHARE_DRAFT_WORKSPACE=1`: Merlin environment switch, in the plan described as the MTP draft sharing the target's workspace; dropped automatically if an error line names it or every pin step fails with it.
- **Measured F**: $F = \lceil F_{\log} - 0.05 \rceil$ with $F_{\log} = \text{pool}/X - \lceil M/B \rceil$ from the server's own "Maximum concurrency for M tokens per request: Xx" line (fixed blocks per request).
- **Concurrency**: workers = $\min\left(\lfloor \text{pool}/(\lceil 65{,}536/B \rceil + F) \rfloor,\ \texttt{max-num-seqs}\right)$, with `--max-num-seqs` 9 / 8 / 8 / 6 on the four pin steps; the production boot therefore runs up to 9 agent sessions. If the pre-solve isolation check is not PASS, workers drop to 6.
- **Slots admission**: an attempt starts when one of the *workers* slots is free (the engine schedules KV; preemptions are allowed and counted); an attempt whose projected peak exceeds the whole pool never starts. The scheduler's KV gate is set to 1.0. No new attempt starts with less than 300 s to the solve end.

A.7 Run safety

- Clock anchored at kernel start. Budget 12 h − 3 min = 717 min; hard stop at 715 min; solving ends 180 s before it (712 min), every request clamped to that deadline, then a 90-s join grace; the engine cell's watchdog kills live servers at 717 min.
- No separate drain: the drain and admit deadlines of the time block are defined but not used on this path; the per-attempt budget, the deadline clamp and the 300-s admission cutoff end the work.
- Boot ladder: at most 3 engine attempts (30, 15, 15 min ready timeouts); ready = `/health` answers and the server log shows its KV-cache line; 600 s of log silence = stall; any failure before solving moves to the next pin step; no new try with less than 25 min left.
- Server watchdog during solving (10-s poll): a dead process, a silent log with `/health` failing for 180 s, or 600 s without token progress while requests run triggers a relaunch with the same plan (at most 2 watchdog actions, only with ≥ 20 min left, and ≥ 35 min before the solve end; ready wait ≤ 20 min; a sampled check must pass). Attempts wait while the server is down.

- Submission: the identity `submission.json` is written at the start; it is rewritten atomically after every finished attempt (until the solve end) and once more at the end.
- The sandbox self-test and the memory-limit test must pass, else the solver does not run and the identity file stays.

A.8 The 3-bit "klee100" build (submission 57896498)

Notebook `esfersami50/q27-xhigh-e28-k100-eval120-6h`. Its `RERUN_PROD` is the profile above plus the E28 levers; everything not listed (12 h − 3 min budget, boot ladder, slots admission, measured F, #60490, shared draft workspace, MTP $K = 3$ probabilistic, max-model-len 131,072, prompt, tools, sampling) is the same.

	W4A16 (57851995)	klee100 (57896498)
Routed experts	W4A16 (5 parts)	AutoRound 3bpw-MTP (`klee100/Qwen3.8-Flash-Next-AutoRound-3bpw-MTP`): W3 g128 on layers 0–11 and 36–47, W2 g64 on 12–35 (humming kernels); MTP experts W4; main projections W8
KV scales	W4 checkpoint's QSA sidecar	the same W4 sidecar reused (labelled experimental)
Extra flags	–	`--language-model-only`, a read-only worker extension
Pin ladder (KV bytes)	3.55 / 3.29 / 3.02 / 2.48 GB	7.93 / 6.86 / 5.27 / 3.55 / 2.48 GB
Workers	up to 9	24 on the top rung (`--max-num-seqs 24`); lower rungs min(rule, 18/14/9/6)
CUDA graphs	CAPS16 (to 64)	sizes to 84
Q-defer	off	on when its patch verifies (`NB4_PPDEFER=1`): the pipeline stage-0 receive of sampled tokens is posted after the next forward launch
I8	off	`DRAFT_HEAD=int8` (int8 MTP draft LM head), kept only on read-back `I8_ENGAGED`
C12	off (9 workers)	not applicable (C12 = 12 streams, a W4 lever)
Request timeout	3,600 s	7,200 s (A/B1/C); 9,000 s (B2)
Thinking budget	104,000 per attempt	A, B1, C: 104,000; B2: 120,000
Order	ORDER2	plain (sorted task ids)
Scheduler	attempt 1 for all, then attempt 2	A: attempt 1 for all; B: re-run of attempt-1 failures (B1 = ended on its own without a full train match, same budget; B2 = cut by budget/context, 120k); C: attempt 2, first tasks still without a train match, then the rest by ascending attempt-1 tokens
Answer slots	selection above	`attempt_1` = best-train candidate of A/B (a full-match B replaces A, the old A answer becomes the fallback); `attempt_2` = C's candidate if different

References

[1] Ekin Akyürek, Mehul Damani, Adam Zweiger, Linlu Qiu, Han Guo, Jyothish Pari, Yoon Kim, and Jacob Andreas. The surprising effectiveness of test-time training for few-shot learning, 2024. arXiv:2411.07279.

[2] Harold Bessis, Jeroen Cottaar, Isaiah Pressman, Andries Smit, Michal Tešnar, and Stefano Viel. Duck harness: Winning solution for ARC-AGI-3 milestone 1, 2026. Tufa Labs, `https://tufalabs.ai/research/duck-harness/`.

[3] Chakravorty, Manik, and Altaner. Agentic coding improves ARC AGI 2 performance across models, 2026. pivotools blog, `https://pivotools.github.io/pivotools-quarto-blog/posts/agentic_coding_arc_agi/`.

[4] Yi-Chia Chen. Public 2nd place solution, 2026. ARC Prize 2026 ARC-AGI-2 discussion 755801, `https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2/discussion/755801`.

[5] François Chollet. On the measure of intelligence, 2019. arXiv:1911.01547.

[6] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2025: Technical report, 2026. arXiv:2601.10904.

[7] François Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems, 2025. arXiv:2505.11831.

[8] Domdolus Tolus. Domdolus tolus solution, 2026. ARC Prize 2026 ARC-AGI-2 writeup and public notebook, `https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2/writeups/Domdolus-Tolus-Solution`.

[9] fit-the-whale team. Fit the whale in: running DeepSeek-V4.1-Flash on 4 L4, 2026. `https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2/writeups/fit-the-whale-in-running-deepseek-v4-1-flash-on-4`.

[10] Daniel Franzen. ARC-AGI-3 solution (fork of the duck harness), 2026. `https://github.com/da-fr/arc-agi-3-solution`, commit 10882e33.

[11] Daniel Franzen, Jan Disselhoff, and David Hartmann. The LLM ARChitect: Solving ARC-AGI is a matter of perspective, 2024. ARC Prize 2024, `https://github.com/da-fr/arc-prize-2024/blob/main/the_architects.pdf`.

[12] Daniel Franzen, Jan Disselhoff, and David Hartmann. Product of experts with LLMs: Boosting performance on ARC is a matter of perspective, 2025. arXiv:2505.07859.

[13] Ryan Greenblatt. Getting 50% (SoTA) on ARC-AGI with GPT-4o, 2024. Redwood Research blog, `https://redwoodresearch.substack.com/p/getting-50-sota-on-arc-agi-with-gpt`.

[14] halt95. Merlin engine (vLLM fork for consumer GPUs), tag merlin-v3.0.0, 2026. `https://github.com/halt95/merlin-engine`.

[15] Md. Hamid Hosen, MD. Foysal Emon Shanto, and Md Esfer Abdus Sami. Decomposing the score of an NVARC-derived pipeline on ARC-AGI-2: Generation, selection, and run-to-run noise, 2026. Companion paper, attached to our ARC Prize 2026 Paper Track writeup.

[16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023. doi:10.1145/3600006.3613165.

[17] Wen-Ding Li et al. Combining induction and transduction for abstract reasoning, 2024. arXiv:2411.02272.

[18] Majkel. 8th public place solution writeup, 2026. `https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2/writeups/8th-public-place-solution-writeup`.

[19] nisshokuitsuki. Public 10th: Qwen3.8-Flash-Next agent on 4 L4 GPUs, 2026. `https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2/writeups/public-10th-qwen3-8-flash-next-agent-on-4-l4-gpu`.

[20] notnitsuj. Scaling Qwen3.8-Flash-Next and Dots3, 2026. ARC Prize 2026 ARC-AGI-2 writeup, `https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2/writeups/solution-scaling-qwen-3-8-flash-next-and-dots-3`.

[21] Poetiq. Poetiq ARC-AGI solver, 2025. `https://github.com/poetiq-ai/poetiq-arc-agi-solver`.

[22] Julien Pourcel, Cédric Colas, and Pierre-Yves Oudeyer. Self-improving language models for evolutionary program synthesis: A case study on ARC-AGI, 2025. arXiv:2507.14172.

[23] Qwen Team. Qwen3.8-Flash-Next model card, 2026. `https://huggingface.co/Qwen/Qwen3.8-Flash-Next`.

[24] Ivan Sorokin and Jean-François Puget. NVARC solution to ARC-AGI-2 2025, 2025. ARC Prize 2025, `https://github.com/1ytic/NVARC`.

[25] Maciej Sypetkowski et al. 9th public LB place writeup, 2026. `https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2/writeups/9th-public-lb-place-writeup`.